

ImageClassification

Dimitrios Pakakis

March 2026

1 Introduction

In this project, i conduct a comprehensive analysis of the behavior of the Perceptron for image classification. Implementing techniques as One vs One and One vs All and using the FashionMNIST dataset,i will experiment with the implementations and hyperparameters and explore the impact of different hyperparameters as well as data preprocessing techniques, like the way to convert images into vector representations, are implemented.

The primary objective is to better understand how these algorithms perform under different configurations and conditions. By evaluating their behavior, i aim to identify ways to optimize their classification performance.

2 Dataset

Fashion-MNIST is a dataset of Zalando's article images, consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28x28 grayscale image, associated with a label from 10 classes. Each image is 28 pixels in height and 28 pixels in width, for a total of 784 pixels in total. Each pixel has a single pixel-value associated with it, indicating the lightness or darkness of that pixel, with higher numbers meaning darker. This pixel-value is an integer between 0 and 255.

3 Evaluation Metrics

For the evaluation i will use:

- Accuracy: Number of correct predictions
- Macro Precision: average for all the classes of how often the model predictions are false (FP)

- Confusion Matrix: It compares the predictions made by the model with the actual results and shows where the model was right or wrong.

For the accuracy and precision 100% means perfect performance.

4 Normalization

Normalization is a technique that improves both the training time and often the performance of algorithms. We will use Min-Max scaling in this project.

5 Implementing Basic Perceptron

Perceptron is a linear classifier, where the prediction given a vector $\vec{x}$ is: $\vec{w}^T \vec{x} + b$. After the prediction, using the step function (if positive : 1 else 0) we calculate the perceptron's output. Perceptron is a simple classifier, wehre the output is 0 or 1, so we can not calculate the Binary Cross Entropy. The error is calculated as the residual and the update is the error multiplied by the learning rate. The perceptron utilize a kind of stochastic gradient descent. This means that the weights updates for each batch , but it doesnt use the gradients.

5.1 Perceptron Test

We test the perceptron on a small subset of the fashion MNIST, using only the classes 0 and 1. I detected that 15 epochs are the ideal number to achieve the best accuracy.The perceptron achieved 96.42% accuracy. In the training curves plot we can detect 2 increases in the validation loss, maybe because of the stochastic update rule. After the epoch 8 the validation loss remained the same,

reaching the maximum potential of the algorithm. Perceptron is a strong and light algorithm, but its simplicity makes it difficult for complex problems. The fact that the model is punished for values that are not exactly the label (e.g. 0.9/1) is leading the model to overfitting and the simple update which does not contain the gradient and the learning rate makes the a perceptron a difficult algorithm to generalize.

5.2 Plots

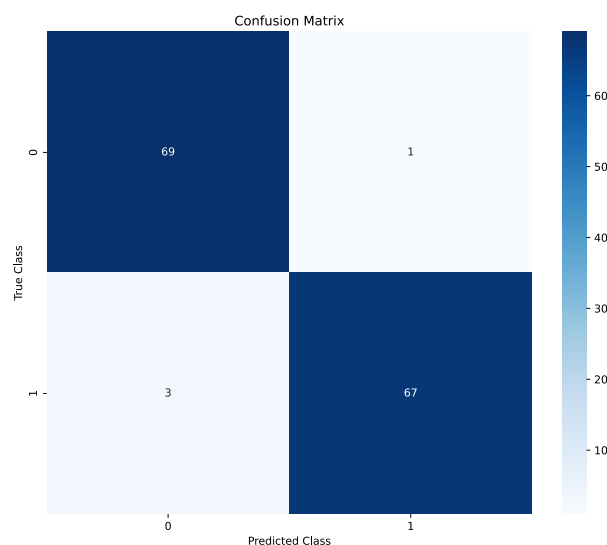


Figure 1: In the figure we can see the small amount of false predictions

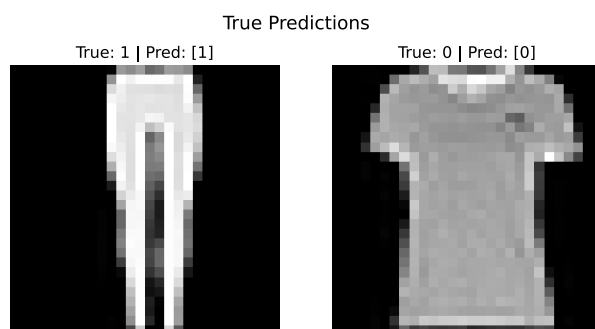


Figure 2: True Predictions

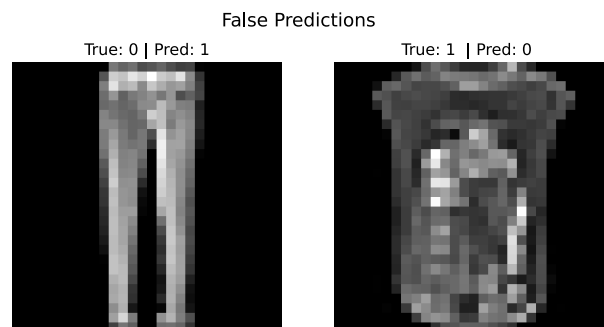


Figure 3: False Predictions

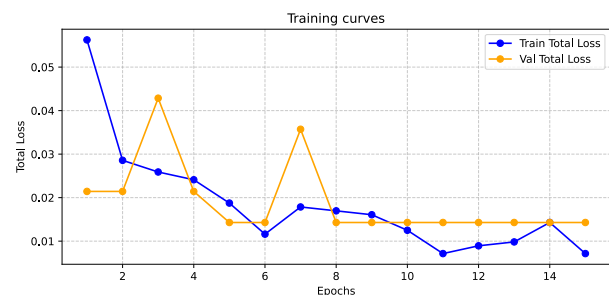


Figure 4: With the learning curves we can detect a small anomaly on the validation loss before epoch 8 and after the validation loss is the same.

5.3 Learning Rate in perceptron

An important observation in the implementation of the Perceptron is the role of the learning rate. Although in machine learning the learning rate is usually one of the most important hyperparameters, in the Perceptron we implemented, it has no significant impact.

This happens because the activation function is not continuous. It simply maps the output to 1 if it is positive, and 0 otherwise. As a result, values like 10 and 0.01 lead to the same final prediction, since only the sign matters.

Even in one-vs-one and one-vs-all implementations, where the z-score is considered, the relative comparisons remain the same, so the effect of the learning rate is still negligible.

Therefore, since we are not using continuous activation functions, and the weights are initialized at zero, the learn-

ing rate does not meaningfully affect either the performance or the learning process of the algorithm.

6.1.2 Plots

6 One vs One using Basic Implementation of Perceptron

The one-vs-one technique requires the creation of $n(n-1)/2$ models, which in our case results in 45 models. Each model is trained to distinguish between two specific classes.

For training, we keep only the data belonging to the two selected classes. One class is assigned as positive and the other as negative. We then train a model to separate these two classes and store it. This process is repeated for all class pairs.

When predicting a new sample, we collect the outputs from all 45 models. Each model “votes” for a class, and the label with the highest number of votes is selected as the final prediction. To avoid ties, we enhance the voting process by adding a small fraction of the confidence score to each vote. This score reflects how confident each model is in its prediction. As a result, ties are eliminated, and the final prediction is determined not only by the number of votes but also by the confidence of the models.

In general, the basic implementation of the Perceptron (residual error, stochastic updates) imposes a performance ceiling. It is a linear classifier, so its performance heavily depends on how the data is distributed in space and how easily it can be separated by a hyperplane.

For now, we will keep this basic implementation (and later introduce more advanced techniques). As a result, there are no hyperparameters to tune, and our analysis will focus primarily on feature extraction.

We will experiment with different representations, including: Flattening, PCA and Average intensity per row and per column.

6.1 Flattening

6.1.1 Results

The One vs One technique using the basic implementation of perceptron achieved accuracy : 78.6% and Macro Precision: 81.3%.

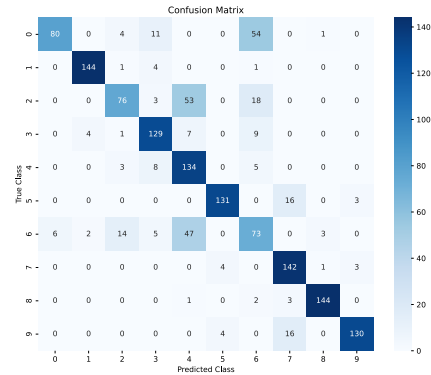


Figure 5: Confusion Matrix

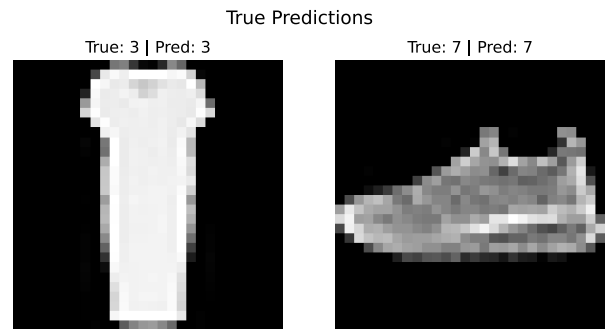


Figure 6: True Predictions

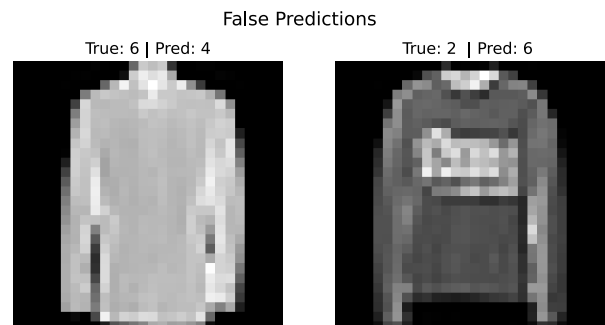


Figure 7: False Predictions

6.1.3 Comments

The training of the 45 models and the prediction process took only 11 seconds. The Perceptron is a very fast algorithm.

Overall, it achieved relatively good accuracy and precision. From the confusion matrix, we observe that the misclassifications are between specific classes and not uniformly distributed. This is a pattern we also noticed in the analysis of distance-based algorithms.

In particular, the errors are concentrated in clothing items with a t-shirt-like structure, where the model struggles to clearly distinguish between similar classes.

6.2 Average intensity per row and per column

6.2.1 Results

The One vs One technique using the basic implementation of perceptron with mean brightness per axis achieved accuracy : 79.9% and Macro Precision: 80.9%.

6.2.2 Plots

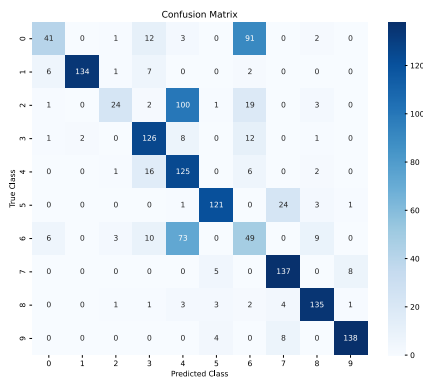


Figure 8: Confusion Matrix

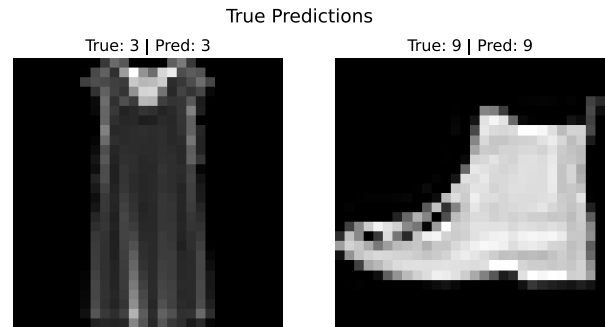


Figure 9: True Predictions

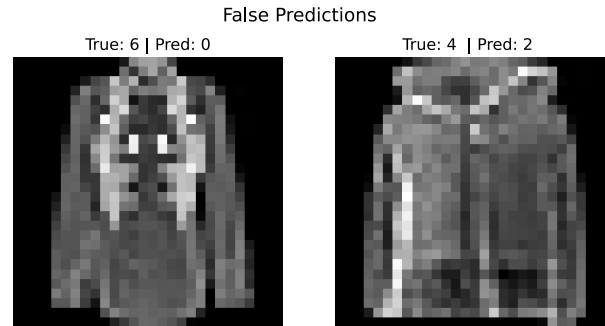


Figure 10: False Predictions

6.2.3 Comments

The accuracy increased slightly, meaning we made more correct predictions overall. However, the macro precision decreased, which indicates that the proportion of incorrect predictions has increased.

It seems that this approach makes certain classes more prominent, which may cause the model to become overconfident. This could explain why the number of false predictions has increased, even though overall accuracy improved.

6.3 PCA

We will use PCA with $n_{components} = 0.9$.

6.3.1 Results

The One vs One technique using the basic implementation of perceptron with PCA achieved accuracy : 80% and Macro Precision: 80.7%.

6.3.2 Plots

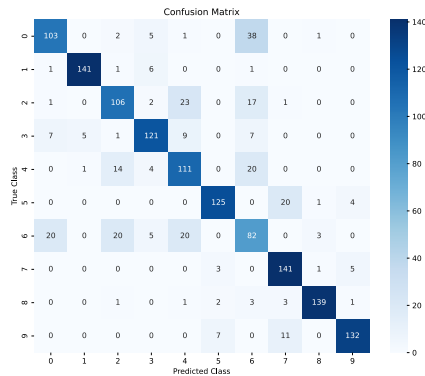


Figure 11: Confusion Matrix

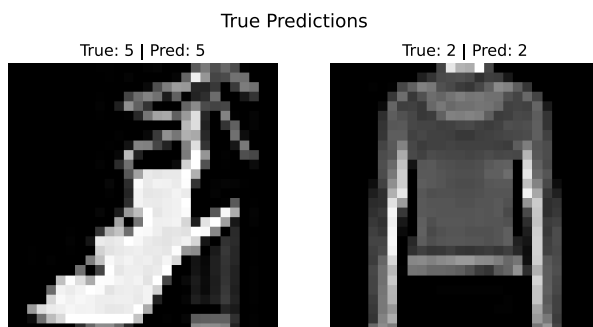


Figure 12: True Predictions

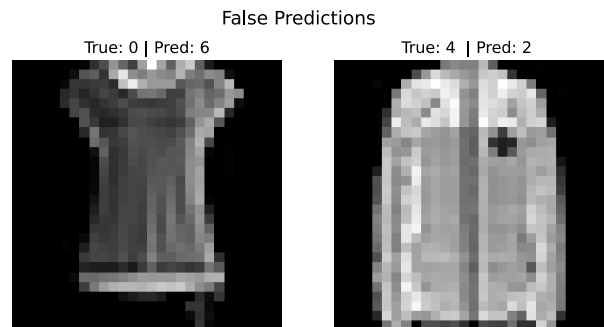


Figure 13: False Predictions

6.3.3 Comments

With the use of PCA, we increased the accuracy to 80%, and from the confusion matrix, it appears that the errors are now more uniformly distributed.

This suggests that, with PCA, the algorithm was able to better distinguish between certain classes, even when they are visually similar.

6.4 One vs One Comments

The one-vs-one technique allows us to use binary classifiers in multi-class problems. Using the Perceptron algorithm together with PCA, we managed to achieve an accuracy of 80%.

Although this approach required 45 models, the algorithm remains very fast, especially compared to KNN. Even without applying further improvements to the Perceptron, and working with the FashionMNIST dataset, we were able to reach a reasonably good level of accuracy.

Now, let us examine whether using the one-vs-all technique can help us surpass the 80% accuracy.

7 One vs All using Basic Implementation of Perceptron

The one-vs-all technique creates as many models as there are classes (10 in our case). Each model is trained to distinguish one class from all the others. In other words, for each model, the target class is treated as positive (1), while all other classes are treated as negative (0).

During training, we repeat this process for all 10 classes, meaning that each model becomes “responsible” for identifying its own class against the rest.

When predicting a new sample, we pass it through all 10 models. Each model outputs a score indicating how confidently it believes the sample belongs to its class. We then select the class corresponding to the model with the highest score.

If a model is confident that the sample does not belong to its class, it will produce a negative score, which will not be selected. Among the models that produce positive scores, we choose the one with the highest value.

A key issue with this approach is class imbalance. Since each model is trained with one positive class and all other data as negative, the negative class heavily dominates. This can cause the model to frequently predict the negative class. To address this, we could use class weights to balance the training. In our case, with 10 classes, each model assigns a weight that is roughly 9 times larger to the positive class, ensuring it has a stronger influence during training.

However, for the Perceptron algorithm, we will not balance the classes, and each class will have equal weight. In the comments section of One-vs-All, we will provide a detailed analysis of why using class weights not only fails to improve performance, but actually makes it worse.

We will experiment with different representations, including: Flattening, PCA and Average intensity per row and per column.

7.1 Flattening

7.1.1 Results

The One vs All technique using the basic implementation of perceptron achieved accuracy : 79.4% and Macro Precision: 82.1%.

7.1.2 Plots

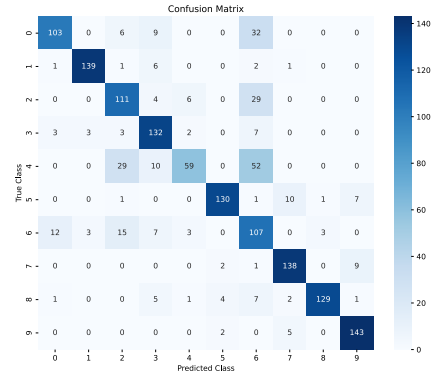


Figure 14: Confusion Matrix

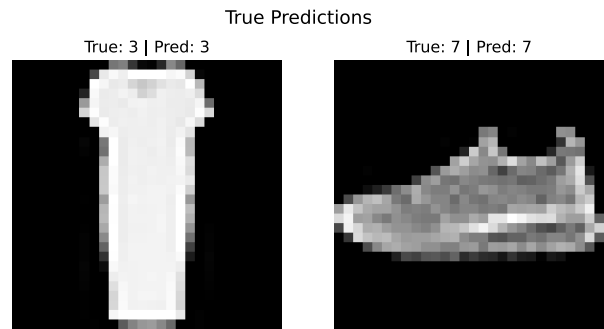


Figure 15: True Predictions

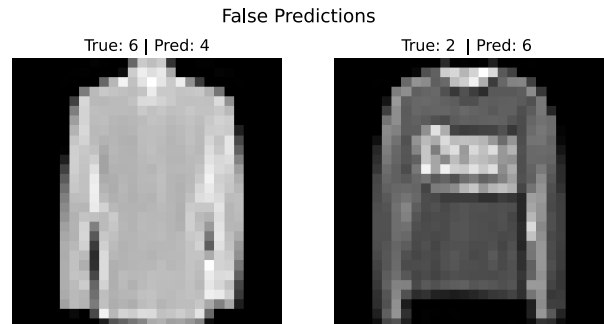


Figure 16: False Predictions

7.1.3 Comments

We achieved good accuracy, and from the macro precision, we can see that the model has improved compared to one-vs-one in making fewer incorrect predictions.

7.2 Average intensity per row and per column

7.2.1 Results

The One vs All technique using the basic implementation of perceptron with mean brightness per axis achieved accuracy : 69.8% and Macro Precision: 75%.

7.2.2 Plots

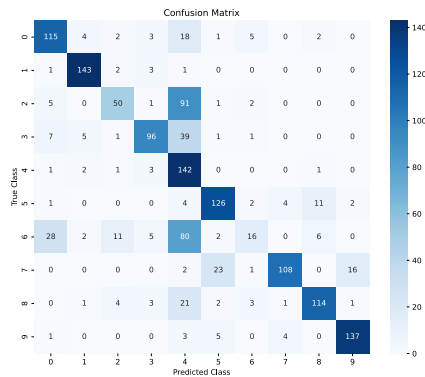


Figure 17: Confusion Matrix

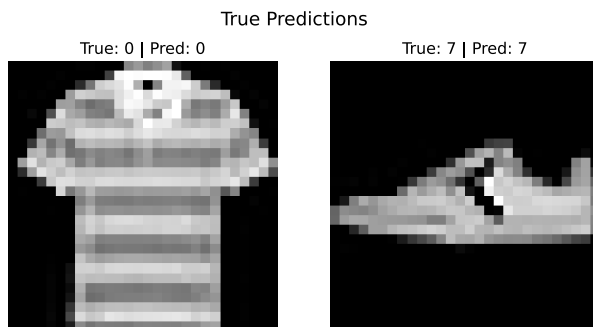


Figure 18: True Predictions

False Predictions

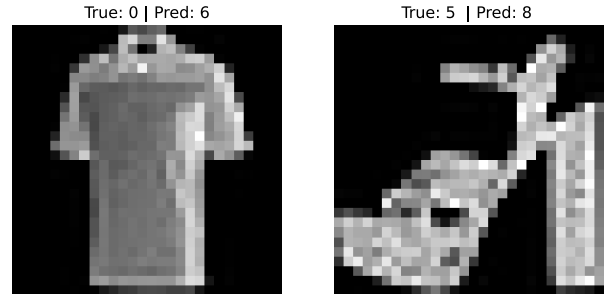


Figure 19: False Predictions

7.2.3 Comments

Both accuracy and macro precision decreased significantly. Although this feature extraction technique slightly improved performance in the one-vs-one approach, here it led to a notable decline.

This suggests that Perceptron models with this type of feature representation are not confident enough in predicting the correct class. As a result, some models become more confident about similar classes, producing higher scores, which then dominate the final decision.

In contrast, in the one-vs-one approach, the presence of many models helped mask this uncertainty, as the final prediction was based on multiple votes, making it more robust.

7.3 PCA

We will use PCA with $n_{components} = 0.9$.

7.3.1 Results

The One vs All technique using the basic implementation of perceptron with PCA achieved accuracy : 78.6% and Macro Precision: 78.4%.

7.3.2 Plots

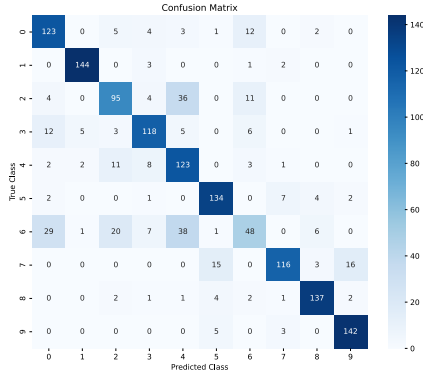


Figure 20: Confusion Matrix

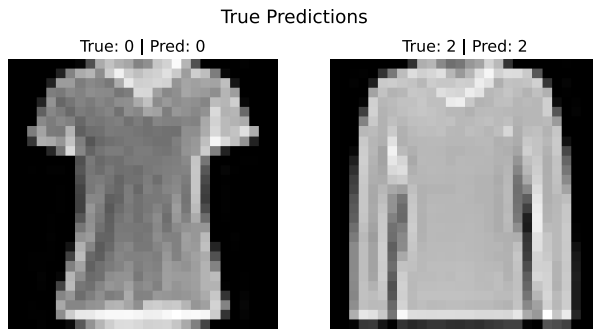


Figure 21: True Predictions

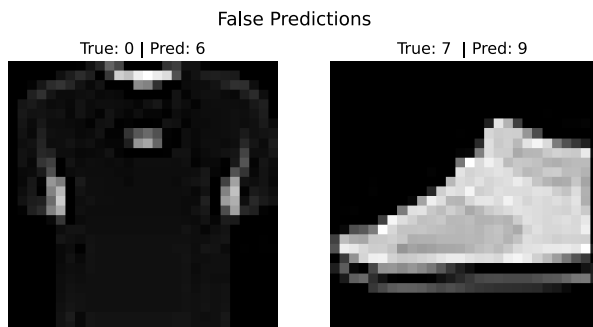


Figure 22: False Predictions

7.3.3 Comments

With the use of PCA, we managed to improve performance compared to the second feature extraction method. However, we were not able to surpass the performance of one-vs-all with flattening. If we used PCA with 95% variance retrieval, the accuracy is 79.2% and macro precision is 80.1%.

7.4 One vs All Comments

So, we can conclude that one-vs-all requires the image representation to retain as much information as possible in order to form more confident predictions.

Although PCA improved performance both in the analysis of distance-based algorithms and in this case, it still did not outperform flattening here. However, using 95% variance retention led to better results, which supports our assumption: the more information we preserve, the more confident the predictions, and the better the performance.

Additionally, the one-vs-all technique outperformed one-vs-one in terms of false positives, meaning it made fewer incorrect predictions, as reflected by the higher macro precision.

However, the 80% accuracy achieved by one-vs-one was not surpassed by the one-vs-all approach.

7.5 Class Weights

In the introduction of one-vs-all, we mentioned that although class imbalance is present, we would not use class weights during training. However, if we assign a weight of 1 to class 0 and a weight of 9 to class 1, we theoretically address this imbalance.

Using our evaluation metrics, we can now observe how performance changes before analyzing the underlying reasons.

With flattening and no class weights, we had: Accuracy:79.4%, Macro Precision:82.1%. With the use of class weights, we achieved: Accuracy:78.2%, Macro Precision:80.5%.

With Mean Brightness per axis and no class weights, we had: Accuracy:69.8%, Macro Precision:75%. With the use of class weights, we achieved: Accuracy:56.5%, Macro Precision:72%.

With PCA and no class weights, we had: Accuracy:78.6%, Macro Precision:78.4%. With the use of class weights, we achieved: Accuracy:75.2%, Macro Precision:77.5%.

If we analyze this behavior more carefully, we can identify two main reasons:

1. Lack of convergence in non-linearly separable data: The Perceptron does not converge when the data is not linearly separable (as in the case of compressed features like the 56-pixel representation). When we introduce class weights, we create significant instability. Each mistake on a positive sample causes a large update in the weight vector. Since the dataset is not separable, the decision boundary never stabilizes and ends up with an almost random orientation.

2. The scores become unreliable: The scores (z-values) of the Perceptron are not probabilities. When we use weights like [1, 9], the model's weights grow in an uneven way. This makes the scores noisy and hard to compare.

As a result, when we choose the final prediction (with `argmax`), we may end up selecting the model with the largest weights, not the one that actually makes the best prediction.

By keeping the weights [1, 1], the scores stay more balanced and comparable, which leads to better results.

8 Multi-Class Perceptron

A multi-class Perceptron works similarly to a binary Perceptron, but instead of having a single weight vector, it has one weight vector per class. In other words, it maintains a weight matrix, where each row corresponds to a different class.

Training Phase: During training, when the model makes a wrong prediction, only two weight vectors are updated: The weights of the correct class are increased, so the model becomes more confident when it sees similar features. The weights of the incorrect predicted class are decreased, so it produces a lower score for those features. In this way, the model learns 10 different weight vectors (for 10 classes), where each one gives a high score only for its own class.

Prediction Phase: When a new image is given, we multiply it with the weight matrix. This produces 10 different scores (one for each class). We then select the class with the highest score as the final prediction.

8.1 Flattening

8.1.1 Results

The Multiclass Perceptron achieved accuracy : 80% and Macro Precision: 80.8%.

8.1.2 Plots

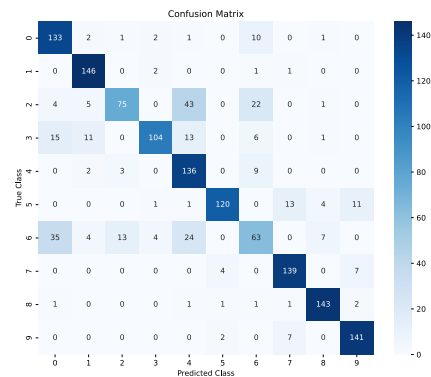


Figure 23: Confusion Matrix

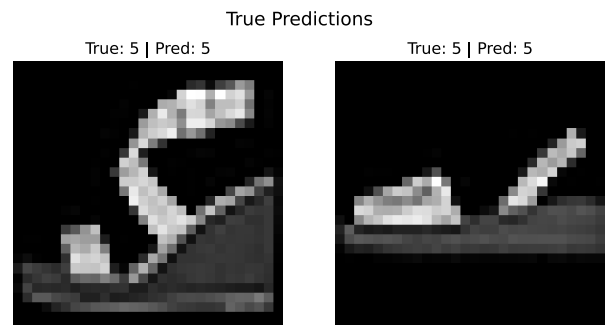


Figure 24: True Predictions

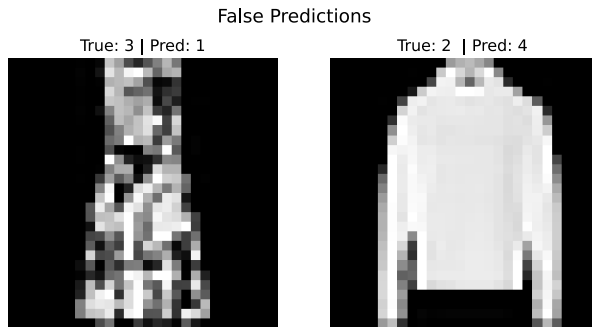


Figure 25: False Predictions

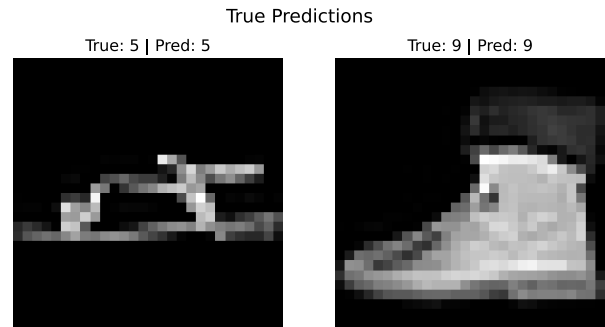


Figure 27: True Predictions

8.1.3 Comments

We managed to reach the 80% accuracy from the one vs one technique. From the confusion matrix we can see that the false predictions are spread across the classes and not in particular classes as we saw before.

8.2 Average intensity per row and per column

8.2.1 Results

The Multiclass Perceptron with mean brightness per axis achieved accuracy : 73.6% and Macro Precision: 73.3%.

8.2.2 Plots

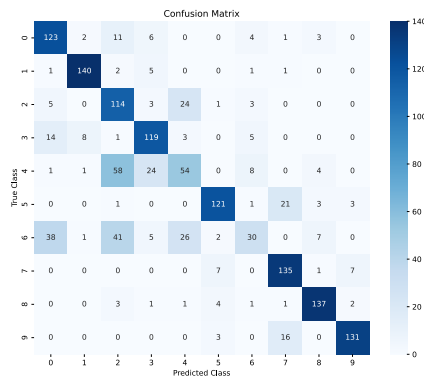


Figure 26: Confusion Matrix

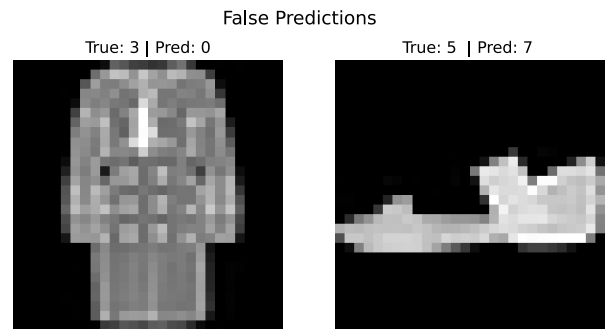


Figure 28: False Predictions

8.2.3 Comments

Here, we observe that both metrics dropped significantly. This feature extraction technique has proven to be ineffective when predictions rely on the model's confidence score.

It is likely that the information it preserves is not enough for the weights of each class to properly adapt, as it tends to represent similar clothing items as identical.

This is also visible from the confusion matrix, where we again observe repeated misclassifications in specific classes.

8.3 PCA

We will use PCA with $n_{components} = 0.95$.

8.3.1 Results

The Multiclass Perceptron with PCA achieved accuracy : 78.4% and Macro Precision: 78.8%.

8.3.2 Plots

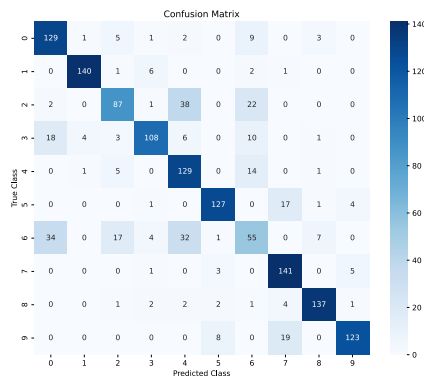


Figure 29: Confusion Matrix

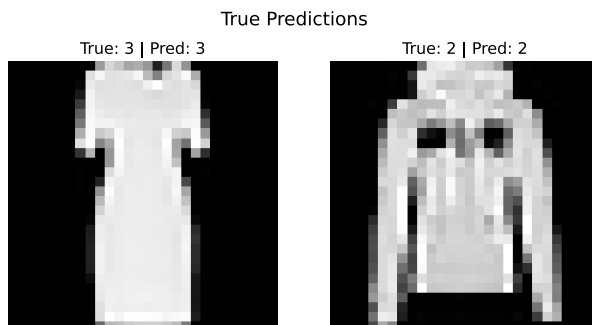


Figure 30: True Predictions

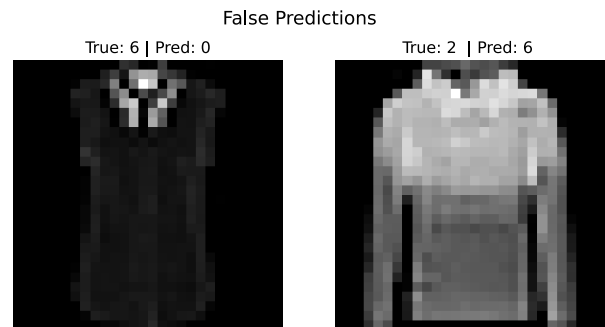


Figure 31: False Predictions

8.3.3 Comments

The performance improved compared to the mean brightness per axis approach, but it did not surpass flattening. Additionally, PCA with 95% variance performs better than PCA with 90% variance.

8.4 Multiclass Perceptron Comments

By using the multiclass Perceptron, we managed to reach an accuracy of 80%, which we were not able to achieve after the one-vs-one with PCA approach. However, the macro precision remains slightly lower from One vs All using flattening.

Additionally, flattening proved to be the best feature extraction method. This shows that when predictions rely on confidence scores, it is very important not to lose information from the images. Preserving more information leads to more confident predictions and better overall performance.

Another important advantage is the computational efficiency. While one-vs-one and one-vs-all required around 10–15 seconds, the multiclass Perceptron needed only 1 second. This is expected, since we train a single model and maintain a weight matrix, instead of multiple separate models.

Moreover, using a multiclass Perceptron allows us to track the training process through training and validation loss (learning curves are included in the notebook). However, updating the weights per batch can lead to unstable validation loss behavior. For example, even if the model is in a good state during an epoch, a few difficult samples at

the end can significantly change the weights. As a result, the validation performance may fluctuate and be strongly influenced by the last training samples.

9 Best Technique

From our analysis, we identified three different techniques with similarly strong performance:

One-vs-One with PCA → 80% accuracy, 80.7% macro precision
One-vs-All with flattening → 79.4% accuracy, 82.1% macro precision
Multiclass Perceptron with flattening → 80% accuracy, 80.8% macro precision

These three approaches show very small differences in performance. Therefore, to determine the best one, we will train all three models on the full dataset and compare their results.

Ideally, we should use cross-validation to ensure a more reliable evaluation, since a specific train-test split might favor one model. However, we will not apply it here, as this is not the main focus of our analysis.

9.0.1 One vs One using PCA

time :1m 34s, Accuracy : 80.7%, Macro Precision : 80.5%.

9.0.2 One vs All with Flattening

time :1m 46s, Accuracy : 82.4%, Macro Precision : 82.9%.

9.0.3 Multiclass Perceptron with Flattening

time :7.2s, Accuracy : 80.9%, Macro Precision : 82.5%.

9.0.4 Comments

It is clear that the second technique (One-vs-All with flattening) outperforms the others, as both accuracy and macro precision are higher. Therefore, we consider this to be the best choice for this problem.

However, the training time of the multiclass Perceptron is noteworthy compared to the other techniques. As explained earlier, this is because it requires only one training process, instead of 10 or 45.

Interestingly, even though One-vs-All involves 10 trainings, its total time is higher than One-vs-One, which uses 45 models. This happens because:

One-vs-All trains each model on the entire dataset
One-vs-One trains each model only on the data of the two relevant classes

As a result, the total computational cost of One-vs-All becomes higher, despite having fewer models.

10 Compare best Technique with Knn

From the analysis of distance-based algorithms, the best performance we achieved was 86.7% accuracy and 86.6% macro precision.

Therefore, we see that we were not able to surpass this performance using the original version of the Perceptron.

However, there is a significant difference in computational cost. The KNN algorithm required 5–7 minutes for prediction, while One-vs-All needed only about 1.4 minutes for both training and prediction.

In conclusion, KNN outperforms the Perceptron in terms of accuracy, but at the cost of higher computational time.

11 Unbalanced Dataset

For each class in FashionMNIST, we will keep 50 samples, while for class 6 we will keep 1000 samples. The choice of class 6 is not random. Throughout our analysis, class 6 has shown a low success rate, mainly because it is visually similar to other classes.

With this setup, we aim to examine whether this intentional imbalance helps the model better recognize class 6 and distinguish it from similar classes, or if it will instead start over-predicting class 6.

In general, when dealing with an unbalanced dataset, we can use class weights to balance the importance of each class. However, as we observed earlier, class weights did not improve the performance of the Perceptron.

So, we will first evaluate how our techniques perform on this imbalanced dataset, and then we can experiment further with class weighting.

We will re-evaluate our techniques, but we will not experiment with feature extraction. The class imbalance does not affect how we extract information from the images.

Therefore, for each technique, we will keep the feature extraction method that performed best in the previous

analysis. In other words, we will use the optimal feature representation for each technique. For the evaluation, we will use the entire test set provided by FashionMNIST.

11.1 Evaluation

Since our dataset is imbalanced, metrics like accuracy and macro precision do not give a fully reliable picture of performance. For example, if the model correctly predicts all 1000 samples of class 6, but fails on all 450 samples from the other classes, it would still achieve over 50% accuracy, which is misleading. In such cases, we should normally use metrics that take class distribution into account, such as: Micro Recall, Micro Precision, Micro F1-score

However, in our case, the evaluation is performed on the original FashionMNIST test set, where the classes are balanced. Therefore, we do not need to introduce different metrics, and our current evaluation approach remains valid.

11.2 One vs One

11.2.1 Results

Accuracy: 63.7%, Macro Precision: 77.9%

11.2.2 Plots

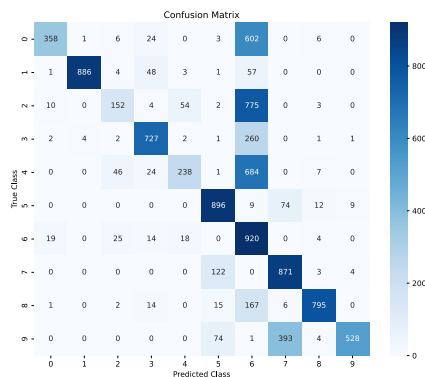


Figure 32: Confusion Matrix

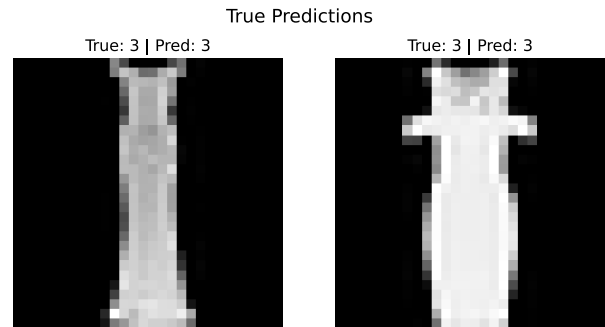


Figure 33: True Predictions

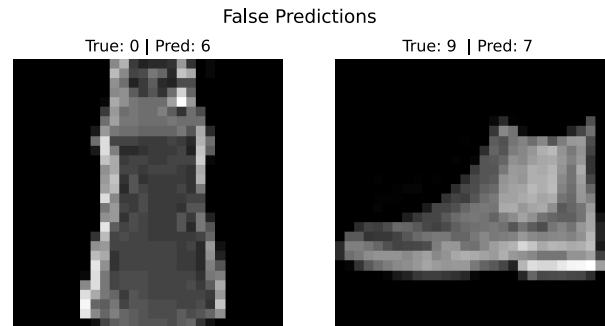


Figure 34: False Predictions

11.2.3 Comments

We observe a large drop in accuracy, but a much smaller decrease in macro precision.

The drop in accuracy happens because many predictions are now incorrect—the model often predicts class 6, even when the true class is different. This is also visible in the confusion matrix, where class 6 dominates the predictions.

On the other hand, macro precision does not decrease as much. This is because macro precision is the average precision across all classes, and precision measures how often the model is correct when it predicts a class.

Due to the imbalanced dataset, the model tends to predict class 6 whenever it is uncertain. This leads to:

Low precision for class 6 (many wrong predictions → many false positives) High precision for the other classes, since the model predicts them only when it is very confident

So, for the other classes, there are few false positives,

which keeps their precision high and prevents macro precision from dropping significantly.

11.3 One vs All

11.3.1 Results

Accuracy: 66.3%, Macro Precision: 78.6%

11.3.2 Plots

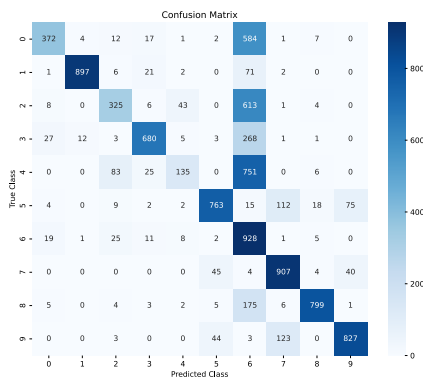


Figure 35: Confusion Matrix

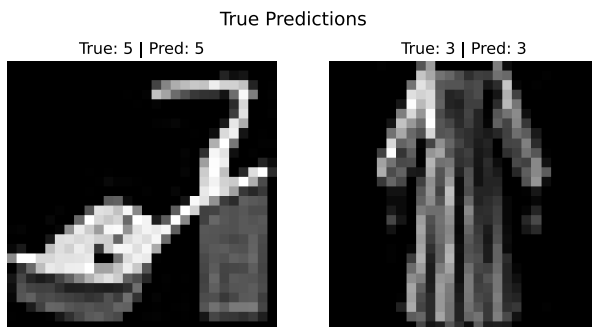


Figure 36: True Predictions

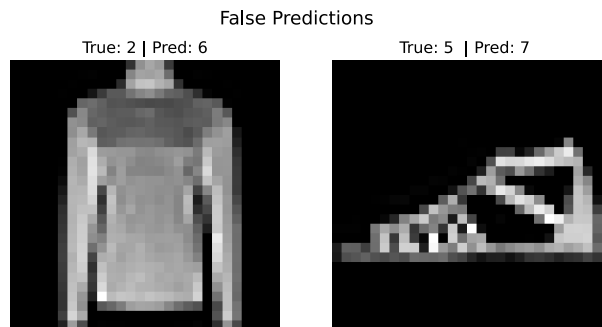


Figure 37: False Predictions

11.3.3 Comments

We observe a similar behavior to one-vs-one. The accuracy decreases, while the macro precision drops only slightly.

The slightly higher accuracy of one-vs-all compared to one-vs-one suggests that the model trained for class 6 vs all other classes is better at recognizing when a sample does not belong to class 6, compared to the pairwise models in one-vs-one.

11.4 Multiclass Perceptron

11.4.1 Results

Accuracy: 63.7%, Macro Precision: 77.9%

11.4.2 Plots

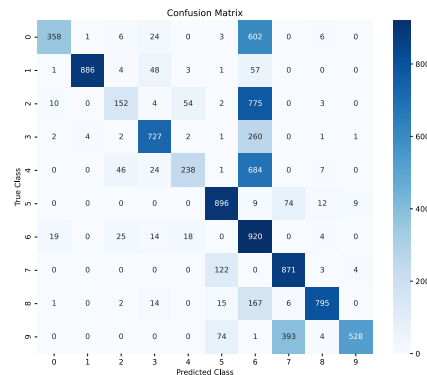


Figure 38: Confusion Matrix

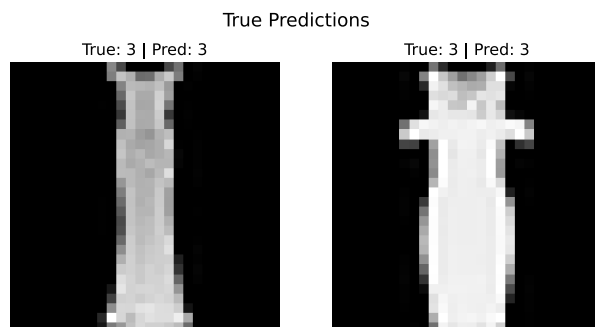


Figure 39: True Predictions

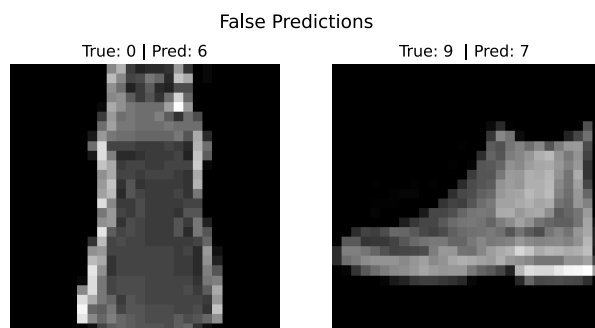


Figure 40: False Predictions

So, for the other classes, there are few false positives, which keeps their precision high and prevents macro precision from dropping significantly.

11.5 One vs All

11.5.1 Results

Accuracy: 64.8%, Macro Precision: 78.8%

11.5.2 Plots

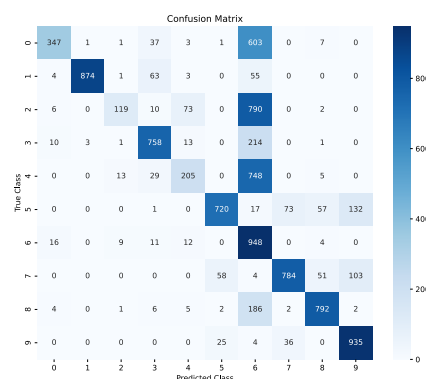


Figure 41: Confusion Matrix

11.4.3 Comments

We observe a large drop in accuracy, but a much smaller decrease in macro precision.

The drop in accuracy happens because many predictions are now incorrect—the model often predicts class 6, even when the true class is different. This is also visible in the confusion matrix, where class 6 dominates the predictions.

On the other hand, macro precision does not decrease as much. This is because macro precision is the average precision across all classes, and precision measures how often the model is correct when it predicts a class.

Due to the imbalanced dataset, the model tends to predict class 6 whenever it is uncertain. This leads to:

Low precision for class 6 (many wrong predictions → many false positives) High precision for the other classes, since the model predicts them only when it is very confident

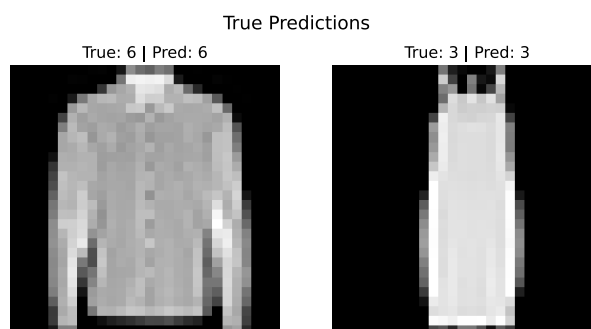


Figure 42: True Predictions

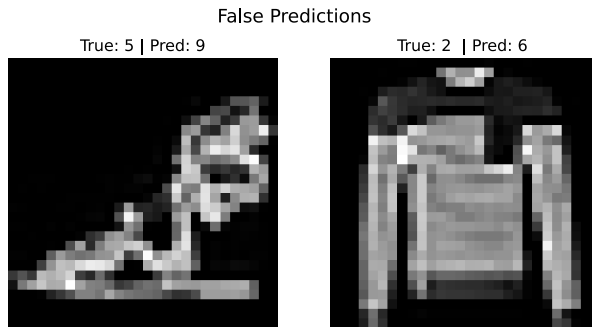


Figure 43: False Predictions

11.5.3 Comments

Here as well, the data imbalance led to similar effects on performance.

11.6 Unbalanced Dataset Comments

It is clear that data imbalance significantly reduced the performance of the algorithms, which is something we expected.

When one class dominates the dataset, models tend to predict that class whenever they are uncertain. In our case, class 6 appears much more frequently during training, so the models adjust their weights in a way that favors predicting class 6.

As a result, the model now makes fewer mistakes for class 6, but it makes many mistakes for the other classes

This is also visible in the confusion matrices, where the column corresponding to class 6 contains many predictions that are not on the main diagonal, meaning many samples from other classes are incorrectly classified as class 6.

Introducing class weights could potentially help address this issue. In one-vs-all, each class was about 9 times less frequent, while here each class is about 20 times less frequent, making the imbalance even more severe and its effects more pronounced.

12 Unbalanced Dataset using class weights

For each class different from 6 we will give a weight of 20, and for class 6 a weight of 1. In this way we want to balance the influence on the weights of each class.

12.1 One vs One

12.1.1 Results

Accuracy: 72.3%, Macro Precision: 74.9%

12.1.2 Plots

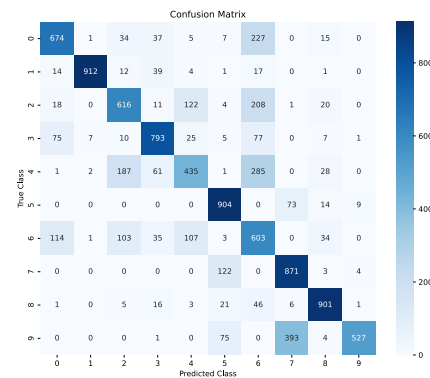


Figure 44: Confusion Matrix

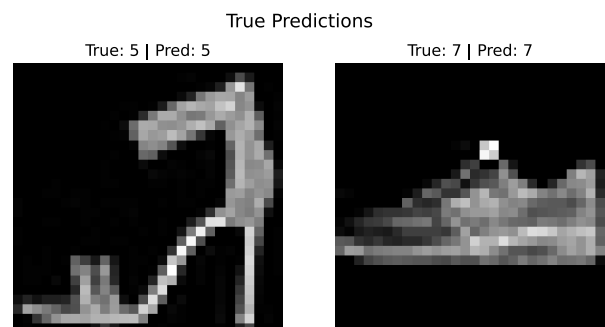


Figure 45: True Predictions

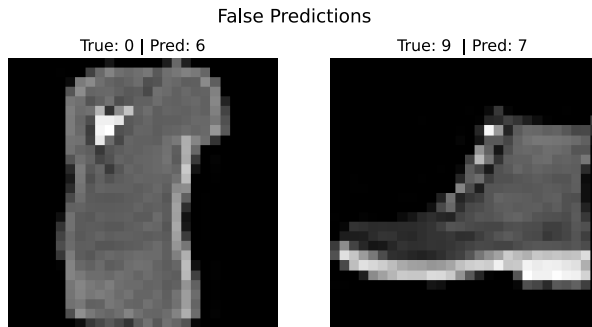


Figure 46: False Predictions

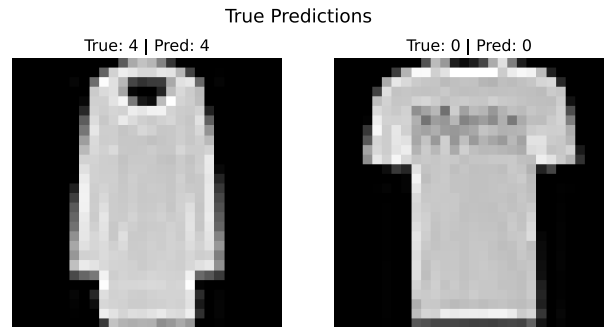


Figure 48: True Predictions

12.1.3 Comments

We observe that by using class weights, we managed to increase accuracy and reduce the model's tendency to incorrectly predict class 6.

However, with weights, the model also becomes more confident in predicting other classes, which leads to an increase in false positives. This is reflected in the slight decrease in macro precision.

12.2 One vs All

12.2.1 Results

Accuracy: 67.5%, Macro Precision: 75%

12.2.2 Plots

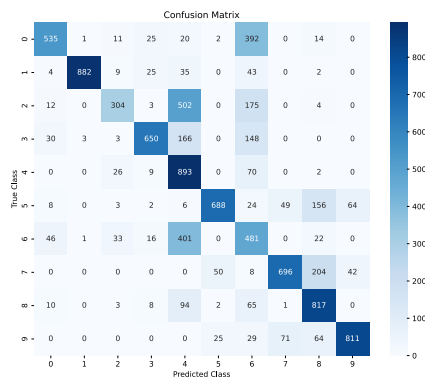


Figure 47: Confusion Matrix

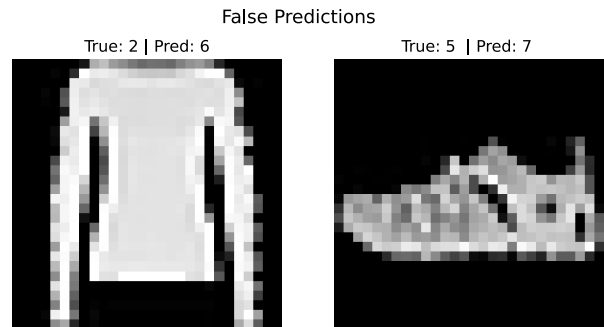


Figure 49: False Predictions

12.2.3 Comments

In one-vs-all, we also observe an increase in accuracy and a decrease in macro precision, but the improvement is smaller compared to one-vs-one.

In general, one-vs-all already introduces imbalance during training. Adding an imbalanced dataset on top of that makes the situation more complex.

For example:

For class 1, we had 50 positive samples and 1400 negative samples → the correct weight should be around 28, not 20. Indeed, using a more appropriate weight increases accuracy by 2–3%. However, for class 6 vs all, we have 1000 positive samples vs 450 negative, so the correct ratio should be around 2.2, not 20 or 28.

This mismatch in weights creates problems. As seen in the confusion matrix, class 6 is no longer predicted correctly and shows many errors.

So, we understand that in one-vs-all, it is not straightforward to apply class weights, because the method already introduces its own imbalance, and combining it with dataset imbalance makes the training process more difficult to control.

12.3 Multiclass Perceptron

12.3.1 Results

Accuracy: 74.7%, Macro Precision: 75.7%

12.3.2 Plots

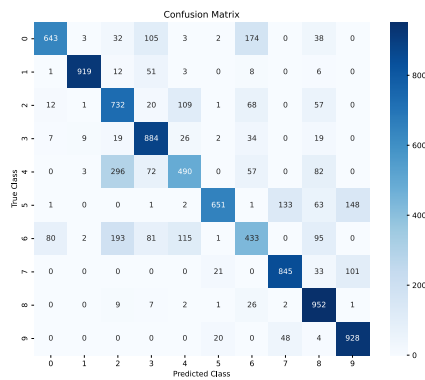


Figure 50: Confusion Matrix

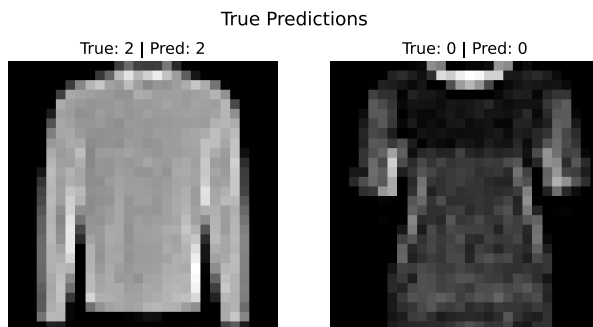


Figure 51: True Predictions

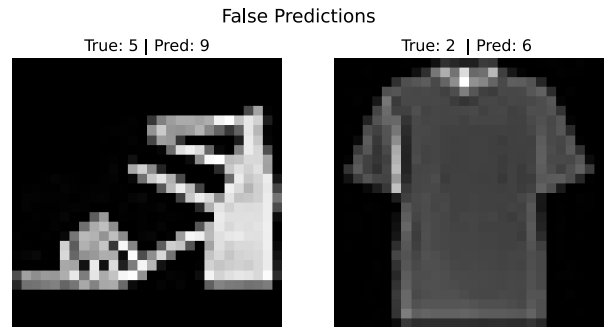


Figure 52: False Predictions

12.3.3 Comments

Here, we achieve a good result, in fact the best among our methods.

One important observation is that during training, we use the same weight factor for both increasing and decreasing the weights. For example, if the true class is 6 and the model predicts 1, the update magnitude is applied equally to both the correct and incorrect classes.

If we did not do this, then in a mistake involving class 6:

The weights of the correct class (6) would increase only slightly While the weights of the other classes would decrease much more (e.g., by 20)

This would make class 6 “win” not because its confidence score is high, but because the scores of the other classes become artificially very low.

It seems that the multiclass Perceptron handles imbalanced data better. A possible reason is that it is a single unified model, where the weight updates are more direct and consistent.

In contrast, techniques like one-vs-one and one-vs-all, which rely on multiple separate models, may introduce additional complexity, making the effect of weights during training harder to control.

12.4 Unbalanced Dataset with Class Weights Comments

We observed that data imbalance significantly reduced the performance of our algorithms, and even after using class weights, we were not able to restore their original performance. This is an expected behavior and is mainly

due to the simplicity of the Perceptron. The algorithm does not allow us to apply more advanced techniques that could better handle imbalanced data, such as loss functions specifically designed for imbalance. Since predictions are based on the confidence score (the dot product between weights and input), using simple class weights introduces instability. The weights get multiplied by large values, causing them to grow excessively and become unbalanced, which negatively affects the model's behavior.

13 Advanced Techniques: Averaged-Pocket Perceptron

Throughout our analysis, and especially in the learning curves, we observe a common pattern: stochastic weight updates introduce instability during training. In practice, this means that the algorithm may be learning well, but if a sequence of difficult samples appears, the weights can change drastically. As a result, the model may “forget” what it had learned, and the final epoch does not necessarily give us the best Perceptron.

To address this, we can apply two important improvements: the averaged Perceptron and the pocket Perceptron. The averaged Perceptron keeps a running sum of the weights after each update and, at the end, divides by the total number of steps. This helps smooth out the updates, reduce the impact of difficult batches, and maintain a more stable learning process.

The pocket Perceptron, on the other hand, keeps track of the model that achieved the lowest validation loss during training. After each epoch, it evaluates performance and stores the best version of the weights. This ensures that we do not lose a good model due to unstable updates later in training.

Our plan is the following: for one-vs-one and one-vs-all, we will use the averaged Perceptron with the same feature extraction methods as before. We will not use the pocket method here, since we are not using a validation set, although it would be possible to do so. For the multiclass Perceptron, we will use both the averaged and pocket versions.

Finally, we will train and evaluate all models on the full FashionMNIST dataset. We will compare them with the best techniques from Section 9 and check whether we

can surpass the 86.3% accuracy achieved by KNN. This will allow us to evaluate whether these improvements can stabilize training and lead to better overall performance.

13.1 One vs One

13.1.1 Results

Accuracy: 84.6% , macro Precision: 84.5%

13.1.2 Comments

Indeed, the averaged Perceptron helped us improve performance from 80.7% accuracy and 80.5% macro precision to the scores shown above. This confirms that the basic version of the Perceptron was indeed harming the training process, and that the averaged version provides a more stable and better-performing model. However, we were still not able to surpass the 86.3% accuracy achieved by KNN.

13.2 One vs All

13.2.1 Results

Accuracy: 84.1% , macro Precision: 83.9%

13.2.2 Comments

In one-vs-all, we also managed to improve performance. The improvement was not as large as in one-vs-one, but it still clearly shows that the averaged Perceptron was beneficial.

This confirms that the averaged version helps stabilize training and leads to better overall results.

13.3 Multiclass Averaged Perceptron

13.3.1 Results

Accuracy: 84.4% , macro Precision: 84.2%

13.3.2 Comments

In one-vs-all, we also managed to improve performance. The improvement was not as large as in one-vs-one, but it still clearly shows that the averaged Perceptron was beneficial.

This confirms that the averaged version helps stabilize training and leads to better overall results.

13.4 Multiclass Pocket Perceptron

13.4.1 Results

Accuracy: 81.1% , macro Precision: 81.2%

13.4.2 Comments

Here we observe that the model with the best validation loss during training is not necessarily the best final model. It may have been optimal for that specific validation set, but it does not always generalize better to unseen data. Therefore, we see that it does not outperform the averaged Perceptron, even though it achieved slightly better performance than the basic implementation in Section 9.

13.5 Multiclass Hybrid Perceptron

If the averaged Perceptron helps us reduce noise and the impact of borderline samples that can destabilize training, and the pocket Perceptron keeps the best model found according to validation loss, then combining the two leads to a hybrid approach.

In this way, we keep the stability of the averaged Perceptron, while also benefiting from the selection ability of the pocket method, which focuses on the model that achieved the best performance during training.

13.5.1 Results

Accuracy: 84% , macro Precision: 83.9%

13.5.2 Comments

Indeed, the performance increased compared to the pocket Perceptron alone, but it did not manage to surpass the performance of the pure averaged Perceptron.

This suggests that the pocket method did not provide a benefit in this setup, and in fact it slightly reduced performance.

If we wanted to be more rigorous, we would use multiple validation sets, and in that case the pocket approach might actually become more effective.

13.6 Advanced Techniques Comments

We saw that with a small modification to the algorithm, we were able to improve its performance. The basic Perceptron caused instability in the weights during training, and with these techniques we managed to smooth the updates and obtain a more stable and better-performing model.

14 Conclusion

In this analysis, we examined in detail how the Perceptron algorithm works, as well as its strengths and weaknesses. Its simplicity makes it a very fast algorithm, both in training and prediction, especially compared to KNN.

We consider the best version we achieved to be the One-vs-One Averaged Perceptron, with accuracy: 84.6% and macro precision: 84.5%. Although it is significantly faster than KNN, it did not manage to surpass its performance.

The Perceptron is the foundation of linear classifiers. A more advanced version of it, which allows the use of more powerful techniques such as better loss functions, gradient descent, and learning rate schedulers, is Logistic Regression.

However, in this analysis, our goal was to keep the classical Perceptron implementation intact and study how it is affected by different techniques and feature representations.